Selenium Automation Framework

Requirements & Architecture Document

A practical Java + Selenium + TestNG automation framework blueprint designed for maintainability, parallel execution, reusable components, reporting, configuration, and CI/CD integration.

1. Requirements Summary

Area	Requirement
Language	Java
UI Automation	Selenium WebDriver
Test Framework	TestNG
Build Tool	Maven
Design	Page Object Model with reusable utilities/components
Execution	Local and CI/CD; parallel execution supported
Browsers	Chrome, Edge, Firefox; browser selection through configuration
Configuration	Externalized properties/environment configuration
Test Data	External test data support with reusable data utilities
Logging	Centralized logging
Reporting	HTML execution reports with screenshots for failures
Artifacts	Screenshots, logs and execution results
Source Control	Git-friendly project structure
CI/CD	Designed for Jenkins/GitHub Actions/Azure DevOps-style pipelines

2. Proposed Architecture

Test Layer → TestNG test classes contain business-level test scenarios and assertions. They should remain thin and delegate browser interaction to page/component classes.

Page Layer → Page Objects encapsulate locators and page-specific actions. Common UI widgets should be represented as reusable components where appropriate.

Core/Driver Layer → DriverFactory and DriverManager handle browser creation, thread-safe WebDriver access, lifecycle and browser configuration.

Utility Layer → Configuration, waits, screenshots, test data, file handling, JavaScript helpers and other cross-cutting functionality.

Reporting/Logging Layer → Centralized reporting and logging provide execution visibility and diagnostic artifacts.

3. Recommended Project Structure

```
src/
```

```
■■■ main/java/
■ ■■■ config/
■ ■■■ driver/
■ ■■■ pages/
■ ■■■ components/
■ ■■■ utils/
■ ■■■ listeners/
■ ■■■ reporting/
■
■■■ test/java/
■ ■■■ tests/
■ ■■■ dataproviders/
■
■■■ test/resources/
■■■ config/
■■■ testdata/
■■■ log4j2.xml

pom.xml
testng.xml
README.md
```

4. Core Components

DriverFactory — Creates WebDriver instances based on browser and runtime configuration.

DriverManager — Maintains thread-local WebDriver instances for safe parallel execution.

BaseTest — Provides common setup and teardown for TestNG tests.

BasePage — Provides common page functionality such as waits, click/type helpers and navigation.

Page Objects — Represent application pages and expose business-oriented actions.

UI Components — Encapsulate reusable widgets such as headers, menus, tables and dialogs.

ConfigReader — Reads externalized properties and environment settings.

WaitUtils — Provides explicit, reusable synchronization methods.

ScreenshotUtils — Captures screenshots on failure or when explicitly requested.

TestDataUtils — Loads and manages external test data.

Listeners — Hooks into TestNG events for reporting, screenshots and lifecycle handling.

5. Design Principles

1. Keep tests readable and focused on business intent.

2. Do not place WebDriver creation logic inside individual tests.

3. Avoid Thread.sleep; prefer explicit waits and condition-based synchronization.

4. Keep locators inside Page Objects/components rather than test classes.

5. Use ThreadLocal WebDriver when enabling parallel TestNG execution.

6. Externalize environment-specific values such as URLs and credentials.

7. Capture screenshots and useful logs automatically when tests fail.

8. Make the framework CI-friendly with command-line parameters and deterministic execution.

9. Prefer composition and reusable components over duplicated page interaction code.

10. Keep utilities generic; application-specific logic belongs in pages/components.

6. Parallel Execution Strategy

Parallel execution should use TestNG's parallel capabilities together with a ThreadLocal. Each executing test thread receives its own browser instance. Driver cleanup must occur in teardown so sessions are not leaked.

Recommended initial approach: parallelize at the test level, validate thread safety, and then increase concurrency based on machine/CI capacity.

7. Configuration Strategy

Use a default properties file for non-sensitive settings such as browser, base URL, timeouts and execution mode. Allow command-line/system-property overrides so CI pipelines can supply environment-specific values without changing source code.

Sensitive credentials should not be committed to Git. Use CI secret variables or an appropriate secret-management mechanism.

8. Reporting and Diagnostics

Every test run should produce a clear result summary. On failure, the framework should capture a screenshot and preserve relevant logs. Reports and artifacts should be written to predictable directories so CI systems can archive them.

9. CI/CD Readiness

The framework should support commands such as Maven test execution with runtime parameters, for example selecting a browser or environment without modifying source code. A CI pipeline can then perform checkout → dependency resolution → test execution → artifact publication.

10. Implementation Roadmap

• Step 1 — Initialize Maven project and dependency management.

• Step 2 — Create package structure and configuration handling.

• Step 3 — Implement DriverFactory and thread-safe DriverManager.

• Step 4 — Implement BaseTest and BasePage.

• Step 5 — Implement reusable waits and common UI utilities.

• Step 6 — Add Page Objects and reusable components.

• Step 7 — Add TestNG listeners, screenshots and reporting.

• Step 8 — Add external test-data support.

• Step 9 — Configure parallel execution.

• Step 10 — Add CI/CD pipeline configuration and documentation.

11. Suggested First Milestone

The first working milestone should prove the framework foundation: Maven builds successfully, a configurable browser starts, a sample page test runs through TestNG, the driver is cleaned up, and a failure produces a screenshot/log artifact. Once this foundation is stable, additional features can be layered on without restructuring the framework.

Document generated for framework planning and implementation. This document is intentionally implementation-oriented so it can be used as the baseline for Step 1.